

Python Static Security Analyzer

Test Case Documentation

Test Case: vulnerable_test2.py

This file is a test case for the main Python Static Security Analyzer project. It is intentionally written with several insecure and unused-code patterns so that the analyzer can be tested against known examples.

The test case is not intended to be used as a real application. Its purpose is to provide input to the security scanner and verify that the scanner correctly identifies the security issues and code-quality issues it was designed to detect.

Usage

Save the following code as `vulnerable_test2.py` and run the main scanner against it:

```
python scanner.py vulnerable_test2.py
```

The scanner analyzes the file statically using Python's Abstract Syntax Tree (AST) and reports the detected findings in the terminal. It also saves the results to `report.txt`.

Purpose of the Test Case

The code contains examples corresponding to the detection rules implemented in the main security analyzer:

- **Hard-coded password:** `password = "123456"` is included to test detection of a hard-coded credential.
- **eval():** Tests detection of a potentially dangerous dynamic execution function, used here to evaluate the string `user_input`.
- **exec():** Tests detection of dynamic Python code execution.
- **os.system():** Tests detection of a system command execution call, invoked here through a variable (`cmd`), which also represents a command-injection risk.
- **subprocess.call():** Tests detection of a subprocess command execution call.
- **SQL injection:** The `get_user()` function builds a SQL query using string formatting (`%s`) with an untrusted name parameter, testing detection of SQL injection risk.
- **Unused variable:** The variable `temp` is included to test the analyzer's unused-variable detection.

Expected Findings

When this test case is scanned, the main analyzer is expected to identify the following categories of findings:

- **HIGH** — Dangerous calls: `eval()`, `exec()`, `os.system()`, and `subprocess.call()`.
- **HIGH** — SQL injection: string-formatted query in `get_user()`.
- **MEDIUM** — Hard-coded secret: the password variable.
- **LOW** — Unused variable: `temp`.

Complete Test Case Code

```
import os
import subprocess
# hardcoded secret
password = "123456"
# used variables
a = 10
b = 20
# unused variable
temp = 999
# dangerous eval
user_input = "a + b"
result = eval(user_input)
# dangerous exec
exec("print('Hello from exec')")
# command injection risk
cmd = "echo Hello"
os.system(cmd)
# subprocess usage
subprocess.call(["echo", "Hi"])
# SQL injection example
def get_user(cursor, name):
    query = "SELECT * FROM users WHERE name = '%s'" % name
    cursor.execute(query)
# safe usage (■■■■ ■■■■ ■■■■■)
total = a + b
print(total)
```